Smart Industrial Gas Leakage Detection and Safety System

Table of Contents

1. Introduction

Gas leakage is one of the most common safety hazards in industries, laboratories, manufacturing plants, and storage facilities. Leakage of combustible gases can cause fires, explosions, equipment damage, and serious health risks to workers. Similarly, excessive temperature levels can indicate unsafe operating conditions and may lead to system failures if not detected in time.

To address these challenges, a Smart Industrial Gas Leakage Detection and Safety System was developed using the ATmega32 microcontroller. The system continuously monitors environmental conditions through a gas sensor and a temperature sensor. Based on the sensor readings, the microcontroller determines whether the environment is operating under safe, warning, or dangerous conditions.

When abnormal conditions are detected, the system automatically activates visual and audio alerts and starts an exhaust fan to improve ventilation. An emergency stop button is also provided to allow immediate shutdown of all outputs during critical situations.

The complete system was designed and tested in SimulIDE Design Suite, while the control logic was implemented using Embedded C programming. The project demonstrates how embedded systems can be used to improve industrial safety through continuous monitoring and automatic response mechanisms.

2. Objectives

The main objectives of this project are:

- To design and develop an automated gas leakage detection and safety monitoring system.
- To monitor gas concentration levels using the MQ-2 gas sensor.
- To measure surrounding temperature using the LM35 temperature sensor.
- To process sensor data using the ATmega32 microcontroller.
- To provide visual indications using Green, Yellow, and Red LEDs.
- To generate warning sounds through a buzzer during hazardous conditions.
- To automatically control an exhaust fan for ventilation.
- To implement an emergency stop feature using an external interrupt.
- To test and validate the complete system through SimulIDE simulation.

3. Background Information and System Block Diagram

Industrial environments often contain combustible gases and heat-generating equipment. Any leakage of gas or abnormal increase in temperature can create unsafe conditions. Therefore, continuous monitoring is necessary to ensure worker safety and equipment protection.

The proposed system follows a simple Input–Process–Output architecture.

Input Layer

The system uses two sensors:

MQ-2 Gas Sensor

The MQ-2 sensor is used to detect combustible gases and smoke. It provides an analog voltage that varies according to the concentration of gas present in the environment.

LM35 Temperature Sensor

The LM35 is a precision temperature sensor that provides an output voltage directly proportional to temperature. It produces approximately 10 mV for every degree Celsius.

Processing Layer

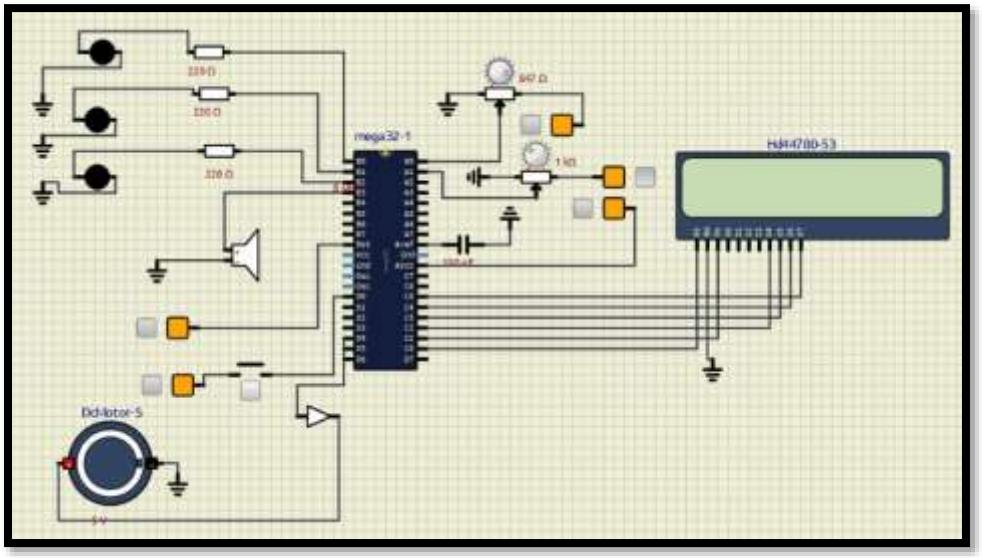
The ATmega32 microcontroller serves as the central processing unit of the system. It continuously reads analog signals from both sensors through its built-in 10-bit ADC. The obtained values are compared with predefined threshold levels to determine the current system state.

Output Layer

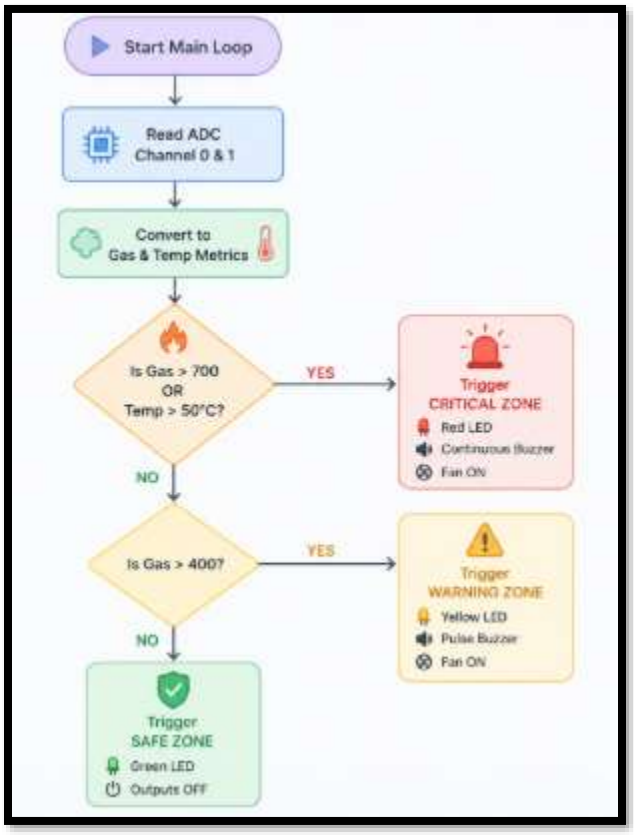
Depending on sensor readings, the following output devices are activated:

- Green LED for Safe condition
- Yellow LED for Warning condition
- Red LED for Dangerous condition
- Buzzer for audible alerts
- Exhaust Fan for ventilation
- 16×2 LCD for status display

System Circuit:



System Flowchart:



4. Materials and Methods

4.1 Hardware Bill of Materials

Component	Quantity	Purpose
ATmega32 Microcontroller	1	Main controller
MQ-2 Gas Sensor	1	Gas detection
LM35 Temperature Sensor	1	Temperature measurement
16×2 LCD	1	Status display
Green LED	1	Safe indication
Yellow LED	1	Warning indication
Red LED	1	Danger indication
Buzzer	1	Audio alert
DC Fan	1	Ventilation
Push Button	1	Emergency stop
220Ω Resistors	3	LED current limiting
10kΩ Resistor	1	Pull-up resistor

4.2 System Hardware Pin Mapping

Component	MCU Pin
MQ-2 Sensor	PA0 (ADC0)

LM35 Sensor	PA1 (ADC1)
Green LED	PB0
Yellow LED	PB1
Red LED	PB2
Buzzer	PB3
Fan	PD4
Emergency Button	PD2 (INT0)
LCD RS	PC0
LCD EN	PC1
LCD D4-D7	PC2-PC5

4.3 Methodology

The project was implemented through the following steps:

Step 1: Circuit Design

The complete circuit was designed in SimulIDE. Sensors were connected to Port A, output devices to Port B and Port D, while the LCD was connected to Port C.

Step 2: Port Configuration

Input and output pins were configured using Data Direction Registers (DDRs). Sensor pins were configured as inputs while LEDs, buzzer, fan, and LCD pins were configured as outputs.

Step 3: ADC Configuration

The ATmega32 ADC module was initialized using AVCC as the reference voltage. A prescaler value of 128 was selected to obtain stable analog readings.

Step 4: Interrupt Configuration

The emergency push button was connected to INT0. Whenever the button is pressed, the interrupt service routine is executed immediately.

Step 5: Program Development

The complete control logic was written in Embedded C. Sensor values were continuously read and compared with predefined threshold values.

Step 6: Simulation and Testing

The generated HEX file was loaded into SimulIDE and tested under different operating conditions to verify system functionality.

5. Observations and Data Collection

5.1 System State Execution Matrix

Gas Value	Temperature	System State	LED	Buzzer	Fan
0–400	≤50°C	Safe	Green	OFF	OFF
401–700	≤50°C	Warning	Yellow	Pulsed	ON
>700	Any	Danger	Red	ON	ON
Any	>50°C	Overheat	Red	ON	ON
>700	>50°C	Critical	Red	ON	ON

5.2 ADC Calculations and Sensor Conversions

The ATmega32 contains a 10-bit ADC, which converts analog voltages into digital values ranging from 0 to 1023.

ADC resolution:

5V / 1023 = 4.887 mV per step

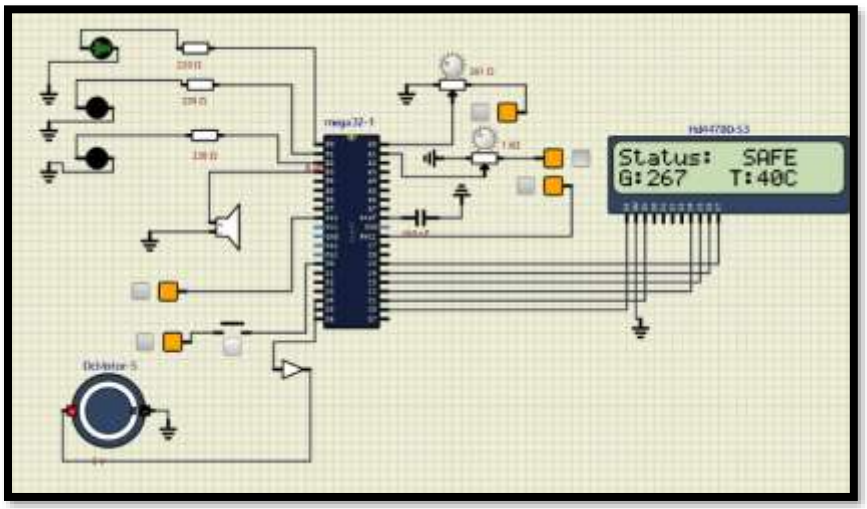
Temperature conversion formula:

Temperature (°C) = (ADC Value × 500) / 1023

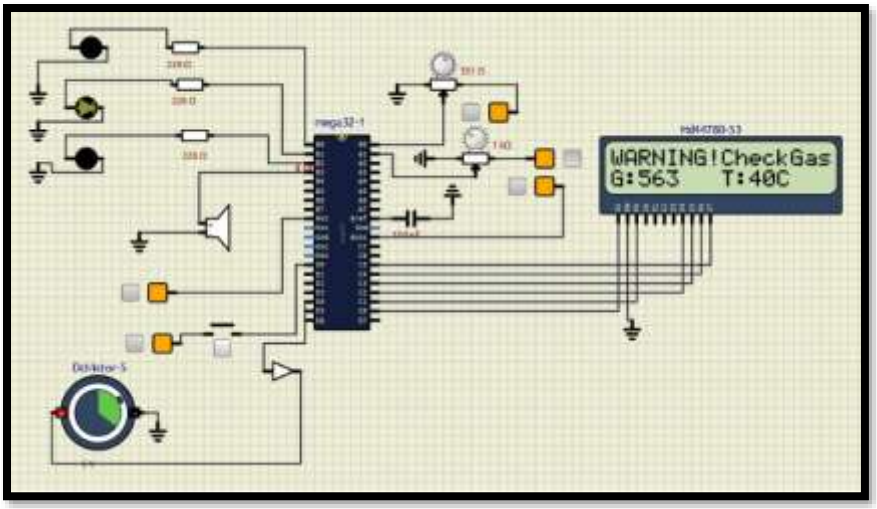
This formula is used because the LM35 produces 10 mV per degree Celsius.

5.3 Simulation Results

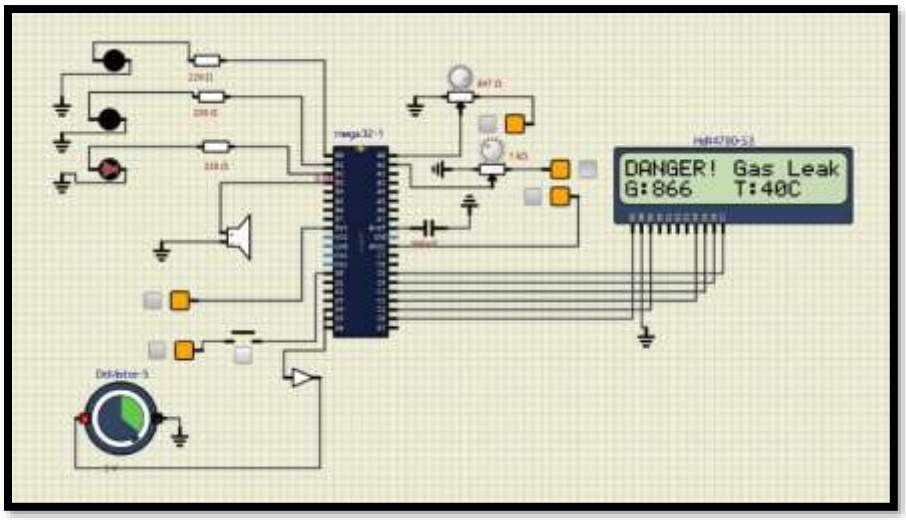
Safe Condition



Warning Condition



Gas Leakage Condition



6. Analysis and Discussion

The developed system successfully detected different environmental conditions and responded according to the programmed safety logic. During testing, the Green LED remained active under normal conditions, indicating safe operation.

When gas levels increased beyond the warning threshold, the system activated the Yellow LED, buzzer, and exhaust fan. This provided an early warning before conditions became dangerous.

When gas levels exceeded the critical threshold or the temperature rose above 50°C, the Red LED, buzzer, and fan were activated immediately. The LCD displayed danger messages, allowing users to identify hazardous conditions quickly.

The emergency stop button also worked successfully. Pressing the button triggered the external interrupt, causing all outputs to shut down immediately regardless of the current system state.

One limitation observed during testing was rapid switching near threshold values. Small fluctuations in sensor readings occasionally caused the system to move repeatedly between states. This issue can be minimized by implementing hysteresis in future versions.

Overall, the system achieved reliable performance and demonstrated the effectiveness of embedded systems in industrial safety applications.

7. Conclusion

The Smart Industrial Gas Leakage Detection and Safety System was successfully designed, implemented, and tested using the ATmega32 microcontroller. The system effectively monitored gas concentration and temperature levels and provided immediate responses when unsafe conditions were detected.

The integration of sensors, LEDs, buzzer, fan, LCD, and interrupt mechanisms enabled the system to operate as a complete safety monitoring solution. Simulation results confirmed that the system accurately detected different environmental conditions and activated the appropriate safety measures.

The project demonstrates the practical application of embedded systems in industrial safety and environmental monitoring.

8. Recommendations

- The following improvements can be implemented in future versions:
- Add hysteresis to reduce rapid switching near threshold values.
- Implement PWM-based fan speed control.
- Store hazard records using EEPROM memory.
- Add GSM or Wi-Fi modules for remote alerts.
- Integrate IoT monitoring for real-time cloud-based data access.
- Add additional sensors for smoke, humidity, and toxic gas detection.

9. Appendix

Production Firmware Source Code

```
#define F_CPU 8000000UL

#include <avr/io.h>

#include <util/delay.h>

#include <avr/interrupt.h>
```

```
#define LCD_RS  PC0
#define LCD_EN  PC1
#define LCD_D4  PC2
#define LCD_D5  PC3
#define LCD_D6  PC4
#define LCD_D7  PC5

#define LED_GREEN   PB0
#define LED_YELLOW  PB1
#define LED_RED     PB2
#define BUZZER      PB3
#define FAN         PD4

/* GLOBAL FLAGS */
volatile unsigned char emergency_flag = 0;

/*
  LCD FUNCTIONS
*/
void lcd_pulse(void)
{
   PORTC |=  (1 << LCD_EN);
   _delay_us(5);
   PORTC &= ~(1 << LCD_EN);
   _delay_us(100);
}
```

```
void lcd_nibble(unsigned char n)
{
   if (n & 0x01) PORTC |= (1 << LCD_D4); else PORTC &= ~(1 << LCD_D4);
   if (n & 0x02) PORTC |= (1 << LCD_D5); else PORTC &= ~(1 << LCD_D5);
   if (n & 0x04) PORTC |= (1 << LCD_D6); else PORTC &= ~(1 << LCD_D6);
   if (n & 0x08) PORTC |= (1 << LCD_D7); else PORTC &= ~(1 << LCD_D7);
   lcd_pulse();
}

void lcd_send(unsigned char val, unsigned char mode)
{
   if (mode)
      PORTC |=  (1 << LCD_RS);
   else
      PORTC &= ~(1 << LCD_RS);
   lcd_nibble(val >> 4);
   lcd_nibble(val & 0x0F);
   _delay_us(100);
}

void lcd_command(unsigned char cmd)
{
   lcd_send(cmd, 0);
   if (cmd == 0x01 || cmd == 0x02)
      _delay_ms(2);
}
```

```
void lcd_char(unsigned char c)
{
   lcd_send(c, 1);
}

void lcd_string(char *str)
{
   unsigned char i = 0;
   while (str[i] != '\0')
   {
      lcd_char(str[i]);
      i++;
   }
}

void lcd_goto(unsigned char row, unsigned char col)
{
   unsigned char pos;
   if (row == 0)
      pos = 0x80 + col;
   else
      pos = 0xC0 + col;
   lcd_command(pos);
}

void lcd_clear(void)
```

```
{
   lcd_command(0x01);
   _delay_ms(2);
}

void lcd_init(void)
{
   DDRC = 0xFF;
   PORTC = 0x00;
   _delay_ms(50);

   lcd_nibble(0x03); _delay_ms(5);
   lcd_nibble(0x03); _delay_ms(1);
   lcd_nibble(0x03); _delay_ms(1);
   lcd_nibble(0x02); _delay_ms(1);

   lcd_command(0x28);
   lcd_command(0x0C);
   lcd_command(0x06);
   lcd_command(0x01);
   _delay_ms(2);
}

/*
  NUMBER TO STRING
*/
void num_to_str(unsigned int num, char *buf)
```

```
{
    unsigned char i = 0;
    unsigned char j = 0;
    char temp[6];

    if (num == 0)
    {
        buf[0] = '0';
        buf[1] = '\0';
        return;
    }
    while (num > 0)
    {
        temp[i] = (num % 10) + '0';
        num = num / 10;
        i++;
    }
    while (i > 0)
    {
        buf[j] = temp[i - 1];
        j++;
        i--;
    }
    buf[j] = '\0';
}

void adc_init(void)
```

```
{
   ADMUX  = 0x40; // VREF = AVCC (5V)
   ADCSRA = 0x87; // Enable ADC, Prescaler = 128
}

unsigned int adc_read(unsigned char channel)
{
   ADMUX  = 0x40 | (channel & 0x07);
   ADCSRA |= (1 << ADSC);
   while (ADCSRA & (1 << ADSC));
   return ADCW;
}

ISR(INT0_vect)
{
   emergency_flag = 1;
}

/*
  OUTPUT HELPERS
*/
void all_off(void)
{
   PORTB &= ~((1<<LED_GREEN)|(1<<LED_YELLOW)|(1<<LED_RED)|(1<<BUZZER));
   PORTD &= ~(1 << FAN);
}
```

```
/*
  MAIN
*/
int main(void)
{
   unsigned int gas_raw  = 0;
   unsigned int temp_raw = 0;
   unsigned int gas_pct  = 0;
   unsigned int temp_c   = 0;
   char str_gas[5];
   char str_temp[5];

   /* GPIO Setup */
   DDRB |= (1<<LED_GREEN)|(1<<LED_YELLOW)|(1<<LED_RED)|(1<<BUZZER);
   PORTB  = 0x00;
   DDRD  |= (1 << FAN);
   DDRD  &= ~(1 << PD2); // Set INT0 pin as input
   PORTD |=  (1 << PD2); // Enable Internal Pull-Up Resistor
   DDRA   = 0x00;        // Set Port A as inputs for sensors

   /* INT0 Setup */
   MCUCR = 0x02; // Falling Edge triggers interrupt
   GICR  = (1 << INT0);
   sei();

   adc_init();
   lcd_init();
```

```
lcd_clear();
lcd_goto(0, 0);
lcd_string("  Gas Detector  ");
lcd_goto(1, 0);
lcd_string("  Starting...   ");
_delay_ms(1000);
lcd_clear();

while (1)
{
   /* 1. Emergency System Interrupt Check */
   if (emergency_flag == 1)
   {
      all_off();
      lcd_clear();
      lcd_goto(0, 0);
      lcd_string("!!EMERGENCY STOP");
      lcd_goto(1, 0);
      lcd_string(" Press to Reset ");

      PORTB |=  (1 << BUZZER); _delay_ms(200);
      PORTB &= ~(1 << BUZZER); _delay_ms(200);
      PORTB |=  (1 << BUZZER); _delay_ms(200);
      PORTB &= ~(1 << BUZZER);

      _delay_ms(1000);
```

```
    emergency_flag = 0;
    lcd_clear();
    continue;
}

/* 2. Read Sensors */
gas_raw  = adc_read(0);   /* PA0 = Potentiometer */
temp_raw = adc_read(1);   /* PA1 = LM35 */

/* 3. Convert Values */
gas_pct = gas_raw;        /* Raw ADC value directly used (0-1023) */
temp_c  = (temp_raw * 500) / 1023; // Validated LM35 Celsius formula

/* 4. Format Strings for Telemetry Display */
num_to_str(gas_pct, str_gas);
num_to_str(temp_c,  str_temp);

/* 5. Clear Pin Register Outputs Before Evaluating New States */
all_off();

/* 6. Independent Zone Logic Structure */

// --- CRITICAL ZONE: High Gas (>700) OR High Temperature (>50C) ---
if (gas_pct > 700 || temp_c > 50)
{
    PORTB |= (1 << LED_RED);  // Red LED ON
    PORTD |= (1 << FAN);      // DC Motor ON
```

```
    PORTB |= (1 << BUZZER);   // Buzzer ON continuously

    lcd_goto(0, 0);
    if (gas_pct > 700 && temp_c > 50) {
        lcd_string("CRITICAL EVACUATE");
    } else if (gas_pct > 700) {
        lcd_string("DANGER! Gas Leak");
    } else {
        lcd_string("DANGER! OVERHEAT");
    }
}

// --- WARNING ZONE: Elevated Gas levels (401 - 700) ---
else if (gas_pct > 400)
{
    PORTB |= (1 << LED_YELLOW); // Yellow LED ON
    PORTD |= (1 << FAN);        // DC Motor ON (Ventilation starts)

    // Pulsing warning beep code:
    PORTB |= (1 << BUZZER);     // Buzzer ON
    _delay_ms(100);             // Short beep duration
    PORTB &= ~(1 << BUZZER);    // Buzzer OFF

    lcd_goto(0, 0);
    lcd_string("WARNING!CheckGas");
}
```

```
        // --- SYSTEM SAFE ZONE ---
        else
        {
            PORTB |= (1 << LED_GREEN);  // Green LED ON
            // Buzzer and Fan stay OFF automatically via all_off()

            lcd_goto(0, 0);
            lcd_string("Status:  SAFE   ");
        }

        /* 7. Bottom Row Real-time Status Data Output */
        lcd_goto(1, 0);
        lcd_string("G:");
        lcd_string(str_gas);
        lcd_string("   T:");
        lcd_string(str_temp);
        lcd_string("C    ");

        _delay_ms(100); // Small delay to handle the loop cycle smoothly
    }

    return 0;
}
```

The complete Embedded C program used for system implementation is attached in this appendix for reference and verification purposes.

9. References

[1] Microchip Technology Inc., *ATmega32 8-bit AVR Microcontroller Datasheet*, Microchip Technology. Available: https://www.microchip.com

[2] Texas Instruments, *LM35 Precision Centigrade Temperature Sensor Datasheet*. Available: https://www.ti.com

[3] M. Mazidi, S. Mazidi, and R. McKinlay, *The AVR Microcontroller and Embedded Systems Using Assembly and C*, 2nd Edition, Pearson Education, 2011.